

SQL Skill Dossier

Contents

1	Scope	3
1.1	Purpose	3
1.2	Primary Reference DBMS	3
1.3	What this dossier includes	3
1.3.1	Core SQL querying (baseline to advanced)	3
1.3.2	Relational design essentials (SQL-engineer level)	3
1.3.3	Performance & tuning (senior differentiator)	3
1.3.4	Transactions & concurrency (must for senior)	4
1.3.5	Production practices (SQL-adjacent, not DBA)	4
1.4	What this dossier intentionally excludes	4
2	Skill Ladder	5
2.1	Junior (Foundations)	5
2.2	Mid (Productive Professional)	5
2.3	Senior (Engineering Mastery)	6
2.4	Expert (Deep Reliability + Optimization + Strategy)	7
3	Concept Map / Index (Reading Order)	8
3.1	Core Querying Foundations (J → M)	8
3.1.1	SELECT Essentials	8
3.1.2	Joins & Relational Thinking	8
3.1.3	Aggregation & Grouping	8
3.1.4	Subqueries & Set Operations	8
3.2	Advanced Querying (M → S)	8
3.2.1	CTEs (WITH)	8
3.2.2	Window Functions	8
3.2.3	Data Types & Built-in Functions (Practical Core)	9
3.3	Relational Design Essentials (J → S)	9
3.3.1	Keys, Constraints, and Integrity	9

3.3.2	Normalization & Modeling Tradeoffs	9
3.4	Performance & Tuning ($M \rightarrow E$)	9
3.4.1	Indexing Fundamentals	9
3.4.2	Query Plans (EXPLAIN)	9
3.4.3	Query Optimization Patterns	9
3.5	Transactions, Concurrency, and Production ($S \rightarrow E$)	10
3.5.1	Transactions & Isolation	10
3.5.2	Locking & Deadlocks (Conceptual + Practical)	10
3.5.3	Production SQL Essentials	10
4	Core Content	11
4.1	SELECT Essentials	11
4.1.1	The SELECT query mental model ($FROM \rightarrow WHERE \rightarrow GROUP BY$ $\rightarrow HAVING \rightarrow SELECT \rightarrow ORDER BY \rightarrow LIMIT$)	11
4.1.2	Filtering with WHERE (predicates, boolean logic, precedence)	11
4.1.3	NULL semantics (three-valued logic)	12
4.1.4	ORDER BY, LIMIT, OFFSET (determinism and pagination basics)	12
4.2	Joins & Relational Thinking	13
4.2.1	Why joins exist (relational model intuition)	13
4.2.2	INNER JOIN vs LEFT JOIN (and when results change)	13
4.2.3	WHERE vs ON in outer joins (critical senior topic)	13
4.2.4	Join multiplicity (why row counts "explode")	14
4.2.5	Anti-joins: finding "missing" relationships (NOT EXISTS)	14

1 Scope

1.1 Purpose

This dossier is designed to help you master SQL as a professional skill from Junior → Mid → Senior → Expert, with a focus on conceptual understanding, decision-making, and real-world correctness (not tool tutorials). It is also interview-ready because you'll be able to explain concepts clearly.

1.2 Primary Reference DBMS

PostgreSQL is the reference dialect and feature set. It will include short "Cross-DB Notes" only when differences matter (MySQL and SQL Server).

1.3 What this dossier includes

1.3.1 Core SQL querying (baseline to advanced)

- SELECT fundamentals: filtering, ordering, limiting
- Joins: inner/left/right/full, anti-joins
- Aggregations: GROUP BY, HAVING, grouping pitfalls
- Subqueries and set operations: IN/EXISTS, UNION/INTERSECT/EXCEPT
- CTEs (WITH): readability and recursion (conceptual)
- Window functions: ranking, running totals, partitions (high-value)

1.3.2 Relational design essentials (SQL-engineer level)

- Relational model fundamentals: keys, relationships, cardinality
- Normalization basics (1NF–3NF) and when to denormalize
- Constraints: primary key, foreign key, unique, check, not null
- Data types: conceptual use, correctness and tradeoffs

1.3.3 Performance & tuning (senior differentiator)

- Query plans (EXPLAIN) at a practical conceptual level
- Index fundamentals: selectivity, composite indexes, covering/index-only concepts
- Common performance anti-patterns: N+1, missing predicates, bad joins, over-fetching
- Reasoning about cost: scans vs index lookups, cardinality intuition

1.3.4 Transactions & concurrency (must for senior)

- ACID concepts applied to real systems
- Isolation levels (practical understanding)
- Locking and deadlocks (conceptual + what causes them)
- Idempotency and safe retries (how SQL behavior impacts application behavior)

1.3.5 Production practices (SQL-adjacent, not DBA)

- Schema migrations (conceptual): backward compatibility, zero-downtime changes (high-level)
- Security essentials: SQL injection principles, least privilege, roles (conceptual)

1.4 What this dossier intentionally excludes

- Deep DBA topics: replication, HA, backups, server tuning, vacuum internals
- Deep planner internals research
- Stored procedures / triggers as a main skill (we may mention them briefly)
- Vendor-specific deep dives beyond essential differences

2 Skill Ladder

Use this ladder to self-assess your current level. A good rule: you "own" a level when you can explain it clearly and apply it consistently in real scenarios.

2.1 Junior (Foundations)

You can:

- Write correct queries using:
 - SELECT, WHERE, ORDER BY, LIMIT/OFFSET
 - basic JOINS (mostly INNER and LEFT)
 - GROUP BY + common aggregates (COUNT, SUM, AVG, MIN, MAX)
- Understand core relational concepts:
 - tables/rows/columns
 - primary keys vs foreign keys (at a basic level)
 - what duplicates are and why they appear
- Handle NULLs reasonably:
 - understand that NULL is not equal to anything
 - use IS NULL / IS NOT NULL
- Do safe data changes:
 - INSERT, UPDATE, DELETE with correct filtering (avoid "update all rows")
- Use basic subqueries and IN (with awareness that NULL can affect results)

Signals you're Junior+:

- You rarely break queries with syntax errors.
- You can explain what a query does line-by-line.

2.2 Mid (Productive Professional)

You can:

- Use advanced querying patterns:
 - EXISTS vs IN (and why EXISTS is often safer)
 - anti-joins (NOT EXISTS) for "find missing" cases
 - set operations (UNION, INTERSECT, EXCEPT) when appropriate
- Build reliable aggregates:
 - correct use of HAVING
 - avoid grouping mistakes (selecting non-grouped columns)
 - handle duplicates and join multiplicity intentionally
- Use CTEs (WITH) for readability and decomposition.

- Use window functions (basic):
 - ROW_NUMBER, RANK, DENSE_RANK
 - SUM(...) OVER (PARTITION BY ...) for running totals
- Understand indexing conceptually:
 - what an index is, how it speeds lookups
 - why indexes can slow down writes
 - basic idea of composite indexes
- Understand transactions at a working level:
 - why transactions exist
 - commit/rollback
 - what "read committed" generally means (conceptually)

Signals you're Mid+:

- You can design queries that stay correct as data grows.
- You can explain why a query returns too many rows (join multiplicity).

2.3 Senior (Engineering Mastery)

You can:

- Diagnose and improve query performance using query plans (conceptual + practical reading):
 - explain EXPLAIN output at a high level (scans, joins, estimated rows)
 - reason about selectivity and cardinality
 - identify missing/wrong indexes and rewrite queries when needed
- Design schemas with integrity and tradeoffs:
 - normalization to 3NF and when to denormalize
 - choose constraints intentionally (PK/FK/unique/check)
 - understand how schema design impacts query complexity/performance
- Handle concurrency and correctness:
 - isolation levels and anomalies (dirty/non-repeatable/phantom reads) at a practical level
 - understand locking and how deadlocks can happen
 - write SQL that is safe under concurrent updates (conceptually)
- Build reliable production queries:
 - pagination patterns and their tradeoffs (offset vs keyset/cursor)
 - avoid N+1 patterns across app ↔ DB boundaries
 - reason about idempotency and safe retries (application + DB)

- Communicate clearly:
 - explain tradeoffs to developers and stakeholders (e.g., "why this query is slow", "why this schema is safer")

Signals you're Senior+:

- You can look at a slow query and propose 2–3 plausible causes and next steps (plan, indexes, rewrite).
- You can design data models that reduce bugs through constraints, not just application logic.

2.4 Expert (Deep Reliability + Optimization + Strategy)

You can:

- Perform advanced performance reasoning:
 - choose composite/covering indexes strategically
 - understand join strategies (nested loop vs hash join) conceptually
 - estimate which part of a query is the bottleneck
- Design for scale:
 - partitioning concepts (why/when)
 - large-data pagination strategies at scale
 - write queries that remain stable and predictable with huge datasets
- Lead DB-related engineering decisions:
 - define SQL standards for teams (patterns, review checklist)
 - plan safe schema evolution (backward-compatible, zero-downtime concepts)
 - advise on tradeoffs: OLTP vs analytics patterns (conceptually)
- Security and governance awareness:
 - least privilege and access control concepts
 - SQL injection threat model and prevention principles (from both SQL and application sides)

Signals you're Expert+:

- You can guide others through performance incidents and propose durable fixes.
- You can anticipate failure modes (locks, load spikes, bad plans) and design around them.

3 Concept Map / Index (Reading Order)

This is the recommended reading order for mastering SQL efficiently. It's organized from core correctness → advanced querying → design → performance → production.

3.1 Core Querying Foundations (J → M)

3.1.1 SELECT Essentials

- SELECT, FROM, WHERE, ORDER BY, LIMIT/OFFSET
- filtering logic, operator precedence
- NULL semantics (IS NULL, three-valued logic basics)

3.1.2 Joins & Relational Thinking

- INNER, LEFT, RIGHT, FULL
- join conditions, join keys
- duplicates and join multiplicity (why row counts "explode")

3.1.3 Aggregation & Grouping

- GROUP BY, aggregates, HAVING
- grouping correctness rules
- distinct vs group, counting pitfalls

3.1.4 Subqueries & Set Operations

- scalar vs table subqueries
- EXISTS vs IN
- UNION / UNION ALL, INTERSECT, EXCEPT
- anti-joins (NOT EXISTS) patterns

3.2 Advanced Querying (M → S)

3.2.1 CTEs (WITH)

- readability and decomposition
- recursive CTEs (conceptual use cases)

3.2.2 Window Functions

- ranking: ROW_NUMBER, RANK, DENSE_RANK
- analytic windows: SUM/AVG OVER, moving totals
- partitions and frames (conceptual)

3.2.3 Data Types & Built-in Functions (Practical Core)

- numeric/text/date-time types (conceptual correctness)
- casting, formatting, common pitfalls (timezone awareness)

3.3 Relational Design Essentials ($J \rightarrow S$)

3.3.1 Keys, Constraints, and Integrity

- primary keys, foreign keys, unique, check, not null
- why constraints prevent bugs

3.3.2 Normalization & Modeling Tradeoffs

- 1NF–3NF basics
- when and why to denormalize
- modeling one-to-many / many-to-many

3.4 Performance & Tuning ($M \rightarrow E$)

3.4.1 Indexing Fundamentals

- what indexes do, selectivity
- composite indexes (order matters conceptually)
- write overhead tradeoffs

3.4.2 Query Plans (EXPLAIN)

- scans vs index scans
- join strategies (high-level)
- cardinality estimation intuition

3.4.3 Query Optimization Patterns

- avoid unnecessary work (filter early)
- reduce row explosion (join strategy)
- remove N+1 at app/DB boundary
- correct pagination strategies (offset vs keyset)

3.5 Transactions, Concurrency, and Production ($S \rightarrow E$)

3.5.1 Transactions & Isolation

- ACID applied
- isolation levels and anomalies (practical)
- safe retries and idempotency

3.5.2 Locking & Deadlocks (Conceptual + Practical)

- common causes
- how to reduce risk via query and transaction design

3.5.3 Production SQL Essentials

- schema migrations (backward compatibility concepts)
- zero-downtime change mindset (high-level)
- security essentials: SQL injection, least privilege (conceptual)

4 Core Content

4.1 SELECT Essentials

4.1.1 The SELECT query mental model (FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT)

[J] A SELECT query retrieves data from one or more tables. At a minimum, it has:

- SELECT (what columns/expressions you want)
- FROM (where the data comes from)

You can then refine the result using WHERE, ORDER BY, and LIMIT/OFFSET.

[M] Even though you write SELECT first, SQL is conceptually processed in this order:

1. FROM (choose source tables / build the row set)
2. JOIN ... ON (combine tables)
3. WHERE (filter rows)
4. GROUP BY (group rows)
5. HAVING (filter groups)
6. SELECT (compute the output columns)
7. ORDER BY (sort final result)
8. LIMIT/OFFSET (return a subset of rows)

This mental model explains many "weird" behaviors, like why you can't use a SELECT alias inside WHERE in many DBs (because WHERE is evaluated earlier).

[S] Put filters in WHERE as early as possible to reduce data processed downstream. Use ORDER BY only when you need deterministic ordering (otherwise results are not guaranteed). Prefer explicit ordering when paginating.

[E] The logical order is not the same as the physical execution plan. The optimizer can reorder operations, but correctness follows the logical model.

Common pitfalls: Assuming results are naturally ordered (they aren't). Using HAVING for row filters that should be in WHERE. Confusing WHERE vs ON in joins (changes results for outer joins).

4.1.2 Filtering with WHERE (predicates, boolean logic, precedence)

[J] WHERE filters rows using conditions (predicates), such as comparisons (=, >, <), pattern matches (LIKE), and membership checks (IN).

[M] SQL boolean logic includes AND, OR, NOT. Operator precedence matters: NOT > AND > OR. Use parentheses to make intent explicit. For example, "A and B or C" is ambiguous; prefer (A AND B) OR C or A AND (B OR C).

[S] Make filters sargable (index-friendly): avoid wrapping indexed columns in functions if possible. Keep predicates explicit and readable; correctness beats cleverness.

[E] PostgreSQL can still use indexes in more cases than you'd expect (expression indexes exist), but don't rely on it casually—design queries for clarity first, optimize when needed.

Common pitfalls: Missing parentheses causing incorrect results. Filtering in the wrong place with outer joins (`WHERE` can turn a `LEFT JOIN` into an `INNER JOIN`). Using `NOT IN` when `NULLs` can appear (often wrong results).

4.1.3 `NULL` semantics (three-valued logic)

[J] `NULL` means "unknown / missing". It is not equal to any value, not even another `NULL`. Use `IS NULL` and `IS NOT NULL`.

[M] SQL comparisons involving `NULL` yield *UNKNOWN*, not true/false. Example: `col = NULL` is neither true nor false; it's *UNKNOWN*, so it won't pass a `WHERE` filter. That's why `col = NULL` is almost always a bug; you want `col IS NULL`.

[S] Prefer `NOT NULL` constraints where appropriate to reduce `NULL` complexity. Be careful with `NOT IN` (subquery) if the subquery can return `NULL`-this can eliminate all rows unexpectedly. Prefer `NOT EXISTS`.

[E] `NULL` handling becomes critical in analytics and outer joins. For reporting, sometimes you use `COALESCE(col, default)` intentionally, but you should understand whether you're replacing "unknown" with a real value.

Common pitfalls: Using `= NULL` instead of `IS NULL`. `NOT IN` with `NULLs` producing surprising empty results. Assuming `COUNT(column)` counts `NULLs` (it doesn't); use `COUNT(*)` to count rows.

4.1.4 `ORDER BY`, `LIMIT`, `OFFSET` (determinism and pagination basics)

[J] `ORDER BY` sorts results. `LIMIT` restricts the number of rows returned. `OFFSET` skips rows (commonly used for pagination).

[M] Without `ORDER BY`, row order is not guaranteed. Pagination with `OFFSET` can get slower for large offsets because the DB may still scan and discard rows.

[S] Always define a deterministic ordering for pagination, usually by a unique column (`id`, or `timestamp + id`). For large datasets, prefer keyset pagination (cursor-based) over offset pagination.

[E] Keyset pagination typically looks like: `WHERE (created_at, id) > (:lastCreatedAt, :lastId) ORDER BY created_at, id LIMIT 50`. This scales better and avoids drifting results when data changes.

Common pitfalls: Paginating without a stable `ORDER BY` (duplicate/missing rows across pages). Using `OFFSET` for very deep pages and expecting good performance. Sorting on non-indexed columns for large datasets without understanding cost.

4.2 Joins & Relational Thinking

4.2.1 Why joins exist (relational model intuition)

[J] A JOIN combines rows from two tables based on a matching condition (usually a key relationship). Joins let you query related data without storing everything in one table.

[M] Conceptually, a join builds a combined row set: it takes rows from table A and table B, applies the join condition (for example, `a.customer_id = c.id`), and returns matching combinations depending on join type.

[S] Join only what you need (select the required columns). Prefer clear join keys and enforce them with constraints when possible. Always validate expected cardinality (one-to-one, one-to-many, many-to-many) to prevent row explosions.

[E] Performance-wise, the optimizer can choose different join strategies (nested loop, hash join, merge join). You don't need internals early, but you must learn to read plans later.

Common pitfalls: Joining tables without understanding cardinality can duplicate rows. Joining without filtering early can create huge intermediate sets.

4.2.2 INNER JOIN vs LEFT JOIN (and when results change)

[J] INNER JOIN returns only rows that match in both tables. LEFT JOIN returns all rows from the left table, plus matching rows from the right; non-matching right columns become NULL.

[M] LEFT JOIN preserves left-side rows even when no match exists. This is how you query "all customers and their orders (if any)".

[S] Choose join type based on the business question: "only those with matches" → inner; "include even those without matches" → left. Be careful: placing conditions in WHERE can turn a LEFT JOIN into an INNER JOIN unintentionally.

[E] In PostgreSQL, the planner can reorder joins, but outer join semantics restrict reordering. Learn to spot when the join type forces plan shape.

Common pitfalls: Using LEFT JOIN but filtering the right table in WHERE drops null-extended rows.

4.2.3 WHERE vs ON in outer joins (critical senior topic)

[J] ON defines how rows match during the join. WHERE filters rows after the join.

[M] With LEFT JOIN, filtering conditions on the right table belong in ON if you still want unmatched left rows.

[S] If you want to preserve left rows, put right-table conditions in ON. If you want to drop unmatched rows, put conditions in WHERE (effectively inner join behavior).

[E] This is one of the most common causes of "why did my LEFT JOIN act like INNER JOIN?" bugs in production.

Common pitfalls: Right-table filters in WHERE eliminate NULL-extended rows and change the result set.

4.2.4 Join multiplicity (why row counts "explode")

[J] If one row in table A matches multiple rows in table B, the result will contain multiple rows for that A row.

[M] One-to-one: 1 match $\rightarrow$ 1 output row. One-to-many: 1 match $\rightarrow$ many output rows. Many-to-many: many matches on both sides $\rightarrow$ potentially huge outputs.

[S] Always predict expected cardinality before joining. If you need one row per entity, consider aggregation (`GROUP BY`), window functions (for example `ROW_NUMBER` to select "latest"), and use `DISTINCT` with caution.

[E] Duplicate explosion is often a symptom of poor join conditions or missing constraints (for example, missing unique keys).

Common pitfalls: Using `DISTINCT` to "fix" duplicates without understanding why they happen, hides bugs and can be expensive.

4.2.5 Anti-joins: finding "missing" relationships (`NOT EXISTS`)

[J] An anti-join finds rows in A that have no match in B.

[M] The safest pattern is `NOT EXISTS (subquery)`. This avoids common `NULL` issues seen with `NOT IN`.

[S] Prefer `NOT EXISTS` over `NOT IN` when the subquery might return `NULL`. Keep anti-join subqueries selective and indexed where possible.

[E] PostgreSQL can optimize `NOT EXISTS` efficiently in many cases, especially with proper indexes.

Common pitfalls: Using `NOT IN (subquery)` can return empty results if the subquery contains `NULL`.